Aalto University
School of Science

Master's Programme in Computer, Communication and Information Sciences

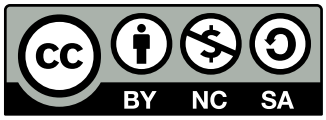
Context is king - Retrieval-Augmented Generation in enterprise-ready systems

Antti-Jussi Asunmaa

Master's Thesis
2026

© 2026

This work is licensed under a Creative Commons "Attribution-NonCommercial-ShareAlike 4.0 International" license.



Aalto University
School of Science

Author Antti-Jussi Asunmaa

Title Context is king - Retrieval-Augmented Generation in enterprise-ready systems

Degree programme Computer, Communication and Information Sciences

Major Computer Science

Supervisor Prof. Petri Vuorimaa

Advisor Lauri Salminen (Msc)

Collaborative partner Nordea Bank Oyj

Date 30 June 2025 **Number of pages** 24 **Language** English

Abstract

The abstract is a short description of the essential contents of the thesis, usually in one paragraph: what was studied and how and what were the main findings.

For a Finnish thesis, the abstract should be written in both Finnish and English; for a Swedish thesis, in Swedish and English. The abstracts for English theses written by Finnish or Swedish speakers should be written in English and either in Finnish or in Swedish, depending on the student's language of basic education. Students educated in languages other than Finnish or Swedish write the abstract only in English. Students may include a second or third abstract in their native language, if they wish.

The abstract text of this thesis is written on the readable abstract page as well as into the pdf file's metadata via the `\thesisabstract` macro (see comment in this TEX file above). Write here the text that goes onto the readable abstract page. You can have special characters, linebreaks, and paragraphs here. Otherwise, this abstract text must be identical to the metadata abstract text.

If your abstract does not contain special characters and it does not require paragraphs, you may take advantage of the `\abstracttext` macro (see the comment in this TEX file below).

Keywords

Aalto University
School of Science

Tekijä Antti-Jussi Asunmaa

Työn nimi Context is king - Retrieval-Augmented Generation is enterprise-ready systems

Koulutusohjelma Computer, Communication and Information Sciences

Pääaine Computer Science

Työn valvoja Prof. Petri Vuorimaa

Työn ohjaaja DI. Lauri Salminen

Yhteistyötaho Yhtiön tai laitoksen nimi (tarvittaessa)

Päivämäärä 30.6.2025 **Sivumäärä** 24 **Kieli** englanti

Tiivistelmä

Tiivistelmä on lyhyt kuvaus työn keskeisestä sisällöstä usein yhtenä kappaleena: mitä tutkittiin ja miten sekä mitkä olivat tärkeimmät tulokset. Suomenkielisen opinnäytteen tiivistelmä kirjoitetaan suomeksi ja englanniksi ja ruotsinkielisen vastaavasti ruotsiksi ja englanniksi. Suomen- tai ruotsinkielisten opiskelijoiden, joiden opinnäytteen kieli on englanti, tulee kirjoittaa tiivistelmänsä englanniksi ja koulusivistyskielellään. Muiden kuin koulusivistyskieleltään suomen- tai ruotsinkielisten tulee kirjoittaa tiivistelmänsä vain englanniksi. Opiskelija voi halutessaan lisätä opinnäytteeseensä toisen tai kolmannen tiivistelmän omalla äidinkielellään. Tämän opinnäytteen tiivistelmäteksti kirjoitetaan opinnäytteen luettavan osan lomakkeen lisäksi myös pdf-tiedoston metadataan. Kirjoita tähän metadataan kirjoitettavaa teksti. Metadatatekstissa ei saa olla erikoismerkkejä, rivinvaiho- tai kappaleenjakomerkkiä, joten näitä merkkeja ei saa käyttää tässä. Jos tiivistelmäsi ei sisällä erikoimerkkejä eikä kaipaa kappaleenjakoa, voit hyödynttää makroa abstracttext luodessasi lomakkeen tiivistelmää (katso kommentti tässä TeX-tiedostossa alla). Metadatatiivistelmatekstin on muuten oltava sama kuin lomakkeessa oleva teksti.

Avainsanat RAG

Preface

I want to thank Professor Pirjo Professor and my instructors Dr Alan Advisor and Ms Elsa Expert for their guidance.

I also want to thank my partner for keeping me sane and alive.

Otaniemi, 30 June 2025

Eddie E. Engineer

Contents

Abstract 3

Abstract (in Finnish) 4

Preface 5

Contents 6

Glossary & Abbreviations 7

1 Introduction 9

2 Technical background 11
2.1 Large-Language Models 11
2.1.1 Evolution and Adoption of Language Models 11
2.1.2 Architectural and Operational Constraints 11
2.2 Naive Retrieval-Augmented Generation 12

3 Literature review 14
3.1 Cascading 14
3.2 Small-to-Big retrieval 16
3.3 Semantic Re-ranking 16
3.4 Prompt Caching 16

4 Methods 17
4.1 Architecture of research program 17
4.2 Research methodology 17

5 Results 18

6 Discussion 19

7 Conclusion 20

Glossary & Abbreviations

Abbreviations

AI	Artificial Intelligence
API	Application Programming Interface
LitM	Lost in the Middle
LLM	Large Language Model
NLP	Natural Language Processing
PLM	Pre-trained Language Model
RAG	Retrieval-Augmented Generation
TTFT	Time-to-First-Token

Glossary

Chunking	The process of dividing a large source text into smaller, distinct segments (chunks) prior to vector database insertion.
Context Window	The maximum amount of text (measured in tokens) that a language model can process in a single prompt iteration.
Cosine Similarity	A mathematical metric used to measure the semantic similarity between two vectors by calculating the cosine of the angle between them.
Embedding	A mathematical vector representation of text that captures its semantic and contextual meaning in a high-dimensional space.
Hallucination	A phenomenon where a language model generates output that is fluent and plausible but factually incorrect or unsupported by the training data.
Inference	The operational phase where a fully trained language model processes a user prompt and generates an output response.
Non-parametric Knowledge	Data or information that resides outside the language model's internal weights, typically stored in an external database, and retrieved dynamically during inference.
Parametric Knowledge	The static, internal information that is permanently embedded within a language model's weights during its training phase.
Time-to-First-Token	A performance metric that measures the latency from the moment a prompt is sent to the model until it begins generating its very first output token.
Token	The foundational atomic unit of text (such as a word, character, or sub-word) into which input strings are split before being processed by a language model.
Vectorization	The algorithmic process of converting textual data into numerical vectors (embeddings) using a machine learning model.

1 Introduction

Modern enterprise-grade AI systems face continuously increasing resource constraints, in addition to growing operational expenditures driven by a rapid acceleration in corporate AI integration and exponential growth in the demand for AI computing capacity [1]. Although API providers frequently lower unit token pricing, overall costs continue to rise due to explosive token usage and the complexity of multi-step agentic workflows that use the latest foundational large language models (LLMs) [2, 3]. This highlights the critical need for optimization across all phases of enterprise AI systems to ensure that computing resources are utilized as efficiently as possible.

This is especially true for Retrieval-Augmented Generation (RAG), which serves as a primary solution to mitigate one of the most critical limitations of modern large language models (LLMs): hallucination [4]. By design, RAG inherently increases token consumption, particularly input tokens, as the core concept relies on prepending retrieved document context to user queries to generate more grounded responses. While various optimization strategies have been proposed in contemporary literature [5, 6], they are typically evaluated on static benchmarks and lack direct integration with industry-standard, production-grade languages and frameworks such as Java and Spring AI [7].

The primary objective of this thesis is to investigate state-of-the-art optimization strategies for RAG systems, and the main contribution is to evaluate how these strategies perform within enterprise environments. Furthermore, this work proposes a concrete architectural solution to implement these optimization strategies utilizing enterprise frameworks. To establish a focused scope, four specific optimization strategies are analyzed in detail: model cascading [2], small-to-big retrieval [8], semantic re-ranking [9], and prompt caching [10].

Each of these methods addresses a distinct phase of the RAG pipeline. Model cascading and small-to-big retrieval optimize the query and retrieval stages, whereas semantic re-ranking refines the retrieved results to minimize prompt size and maximize contextual relevance. Prompt caching, on the other hand, serves to store previous responses in a cache so that subsequent, similar queries can be resolved immediately, thereby bypassing the computationally intensive retrieval and generation pipeline.

Many of these optimization strategies are typically evaluated in isolation, leaving a gap in the literature regarding how they interact when combined. For example, combining semantic re-ranking and small-to-big retrieval could increase the risk of cache misses in prompt caching, thereby reducing its effectiveness. In addition, while model cascading may increase *per-query* latency for complex tasks routed to more capable models, it decreases average latency overall by resolving simpler tasks with faster, less capable models.

Evaluating and resolving these interactive conflicts is crucial for high-volume enterprise applications, such as customer support systems, where operational requirements are exceptionally rigorous. In these environments, minor delays and incremental token

usage quickly compound into systemic instability, elevated latencies, and escalating operational costs. Furthermore, hallucinated or erroneous outputs present substantial risks of reputational damage and potential financial liabilities [11]. Because enterprise systems must strictly navigate these critical constraints, cost, latency, and output quality are established as the primary evaluation metrics for this thesis. To guide this investigation, this study addresses four primary research questions:

1. **RQ1: Latency** - How do small-to-big retrieval, semantic re-ranking, prompt caching, and model cascading affect the latency profile (specifically Time to First Token [TTFT] and overall end-to-end execution latency) of a RAG system?
2. **RQ2: Cost** - To what extent do these optimization methods reduce the operational cost of the system, accounting for token pricing tiers, prompt caching discounts, and model cascading?
3. **RQ3: Quality** - How do these optimizations impact retrieval and generation quality (measured by Context Precision, Context Recall, Faithfulness, and Answer Relevance) compared to a naive RAG baseline?
4. **RQ4: Interaction** - How do these optimization methods interact when combined, and what is the optimal equilibrium between latency, cost, and quality in an enterprise customer support system?

The rest of the thesis is structured as follows: Section 2 briefly explains what LLMs and RAG are, while Section 3 examines the current literature on RAG optimization methods. Research methods are introduced in Section 4, and the results are discussed in Section 5. Finally, the discussion is presented in Section 6, and concluding remarks are provided in Section 7.

2 Technical background

Retrieval-Augmented Generation (RAG) offers an architectural solution to mitigate one of the most fundamental limitations of Large Language Models (LLMs): hallucination. To establish a comprehensive understanding of these concepts, this chapter provides the necessary technical background for both technologies. The first section examines the architectural foundations and inherent drawbacks of LLMs, while the subsequent section focuses on the core principles and operational mechanics of plain vanilla RAG.

2.1 Large-Language Models

2.1.1 Evolution and Adoption of Language Models

Natural language processing (NLP) has traditionally focused on enabling computer systems to understand human-generated language and acquire knowledge [12]. Early statistical approaches, such as n-gram language models, attempted to resolve the inherent ambiguity of natural language by predicting the probability distribution of word sequences [12]. While these traditional models could perform structured tasks like rule-based information extraction or spam filtering, their capability to comprehend semantic nuances and manage complex linguistic tasks remained limited [13].

Many pre-trained language models (PLMs), such as GPT-2 [14], are based on the transformer architecture [15], which excels at handling long-range dependencies through parallel training. While early PLMs required task-specific fine-tuning to perform downstream applications [16], modern large language models (LLMs) scale this architecture to hundreds of billions of parameters. According to Zhao et al. [13], the transition from PLMs to LLMs is primarily characterized by this massive scale in parameters and training data, which gives rise to emergent abilities, enabling the models to solve complex tasks and follow instructions without explicit task-specific retraining.

Presently, large language models are among the most widely used technologies behind modern generative AI applications [13]. Even though these models have demonstrated strong performance across various tasks, such as coding [17], and have been widely adopted[1], they are not perfect.

2.1.2 Architectural and Operational Constraints

Despite their capabilities, LLMs exhibit several major limitations, including biases, hallucinations, and the "Lost in the Middle" (LitM) phenomenon [18, 19]. Bias refers to the fact that the model's output heavily depends on its training data, meaning that the generated content may include inappropriate language, conspiracy theories, or gender bias [18].

[1]For instance, data aggregated in the report [1] highlights a rapid acceleration in corporate AI integration. In Europe, corporate adoption in at least one business function was 91% in 2025 compared to 57% in 2023.

Due to the autoregressive nature of modern LLMs, these models are inherently prone to hallucinations, where they produce outputs that appear plausible but are factually incorrect [20]. This occurs because the models lack an intrinsic understanding of facts, but instead, they generate responses by predicting subsequent tokens based on statistical patterns learned during training. This relying mechanism draws entirely from the model's parametric knowledge, meaning the information is permanently embedded within the model's weights during the training phase [16]. This static nature, consequently, makes it computationally and environmentally expensive and impractical to frequently retrain or fine-tune entire models when the underlying information required to complete a task changes [1].

While the context windows, i.e., LLMs' input sizes, have grown significantly during the last couple of years, allowing models to process up to millions of input tokens [21, 22], these architectures consistently suffer from the LitM phenomenon. This problem refers to a limitation where models effectively utilize information located at the absolute beginning (primacy bias) or the end (recency bias) of the prompt, while failing to retrieve relevant facts buried within the middle sections [19]. In addition, utilizing massive contexts severely increases computational latency and operational expenses, as modern cloud LLM APIs employ token-based pricing models where input costs scale linearly with prompt length and often double when exceeding specific context thresholds [3]. These inherent limitations effectively prevent the full utilization of large context windows in production-grade enterprise environments when attempting to ingest massive enterprise knowledge bases directly into the prompt.

Another source of errors in the output is that the training data has a cutoff date, so it does not contain the most recent information and may generate incorrect answers to questions about recent events [23]. To mitigate these limitations, modern LLM-based systems are integrated with web search capabilities to retrieve real-time information from the internet, a paradigm used, for example, by the Google Gemini model family [21]. While this integration helps keep responses up to date, it introduces an external dependency on the reliability of internet sources. Furthermore, relying purely on open-web search does not solve the challenge for enterprise environments, where the required knowledge is locked behind internal intranet pages and private corporate databases rather than publicly accessible web pages.

2.2 Naive Retrieval-Augmented Generation

To address the limitations detailed in the previous section, Retrieval-Augmented Generation (RAG) offers an architectural solution that decouples the LLM's reasoning capabilities from its static parametrized memory [4]. RAG enriches the LLM's parametric knowledge with non-parametric data served dynamically from an external knowledge base. The LLM's output is based on the user's initial prompt, augmented with the data retrieved from the external knowledge base, and therefore forces the LLM to ground its response in the provided context rather than its parametrized data.

The retrieval-augmentation process requires both the source knowledge and the

user query to be preprocessed. Preprocessing of the knowledge base contains three phases: chunking, vectorizing, and indexing. During this data preparation phase, source documents —such as PDF files— are chunked into smaller, more manageable segments[2]. These chunks are converted into numerical vectors using an embedding model so that their semantic meanings can be mathematically calculated, and they are subsequently stored in an indexed vector database [24]. During runtime inference, the user's initial prompt is vectorized using the same embedding model. The system then calculates the mathematical distance (e.g., cosine similarity) between the prompt vector and the stored vectors, retrieving the most relevant chunks from the database to augment the initial prompt [25].

The primary advantage of the RAG architecture lies in its reliance on an external knowledge base. When properly prompted, this mechanism significantly mitigates hallucinations by forcing the LLM to ground its output exclusively on the retrieved context [26]. Furthermore, maintaining an external data repository is substantially more cost-effective and computationally practical than continuously retraining or fine-tuning the underlying model [24], allowing enterprise systems to remain up-to-date in real time. From a data governance perspective, an external knowledge base ensures that sensitive corporate data remains secure and under internal control [27]. Lastly, by selectively retrieving only the most relevant documents, RAG compresses the required input context, thereby reducing API transaction costs and inference latency compared to naive long-context processing.

However, these benefits introduce distinct trade-offs. The major drawback of a naive RAG pipeline is the increased end-to-end latency of the system [24]. This latency overhead stems primarily from sequential network requests: first querying the embedding model and vector database, and subsequently transmitting the expanded prompt to the LLM. While modern vector distance calculations are highly optimized, the network overhead and the increased time-to-first-token (TTFT) caused by processing larger, augmented prompts add noticeable delay [24]. Consequently, if the RAG pipeline is not properly optimized, the final prompt can grow excessively large, rendering the production system both financially expensive and operationally slow.

[2]Early architectures like Lewis et al. [4] utilized fixed-length segments of 100 words, while modern architectures optimize segment lengths [24].

3 Literature review

3.1 Cascading

Modern RAG systems encounter diverse workloads, ranging from simple informational queries to highly complex analytical tasks. In production-grade enterprise environments —where high availability, low latency, and cost efficiency are critical constraints — a "one-size-fits-all" architecture is fundamentally sub-optimal [28, 2]. It over-allocates expensive computational resources for trivial questions, yet may lack the retrieval depth and reasoning capacity required for complex queries. To mitigate this, model cascading or query routing can be employed to dynamically direct inputs to an appropriately scaled configuration of the RAG pipeline [29]. For instance, a simple query can bypass the full pipeline and utilize a lightweight LLM with minimal context. Conversely, a complex query requiring synthesis across multiple enterprise knowledge bases can trigger an augmented pipeline with extended retrieval top-k parameters and a more capable foundation model.

Section 1: Architectural Benefits, Cost Reductions, and Latency Optimization

* Significant Cost Reductions via LLM Cascading: * Chen et al. [2] demonstrates that an LLM cascade—sequentially querying models from cheapest to most expensive—can achieve up to a 98.3matching the performance of the best individual model (GPT-4). * **?**] reports that learning efficient routers from human preference data can reduce operational costs by over 2 times without sacrificing response quality, achieving a cost-saving ratio of up to 3.66x compared to a fixed GPT-4 baseline. * Xu et al. [30] achieves efficiency by delegating the expensive question decomposition and planning stages to a fine-tuned small language model (SLM) (e.g., Qwen-2.5-3B), which reduces the total token generation burden on proprietary black-box LLMs.

* Latency and TTFT Optimization: * **?**] minimizes latency by bypassing the retrieval pipeline entirely for straightforward queries; the average classification time for a "No Retrieval" decision is just 0.35s, significantly improving overall Time-per-Query metrics. * Guo et al. [31] reports an average wall-clock latency of 0.058s per query in a multi-source setting, demonstrating that a knowledge-routing step adds negligible overhead (0.03s) compared to the gains in retrieval precision. * Jin et al. [32] integrates acceleration libraries like vLLM and FastChat and supports "Conditional Pipelines" to bypass retrieval when internal model knowledge is sufficient, thereby optimizing system throughput and mitigating Time-to-First-Token (TTFT) delays.

* Throughput and Resource Scaling: * **?**] highlights that Matrix Factorization routers are capable of processing over 155 requests per second on a single VM, providing a high-throughput architectural component for multi-model enterprise fleets. * Guo et al. [31] reduces token consumption by up to 9.7x compared to standard agentic RAG frameworks (like Reflexion) by utilizing a "Reasoning Sieve" that isolates retrieval to specific heterogeneous sources rather than processing merged, noisy indices.

Section 2: Routing Mechanisms, Trade-offs, and Judgement Strategies

* Technical Routing Mechanisms: * Scoring-Based Acceptance: Chen et al. [2] utilizes a DistilBERT-based regression model as a scoring function to assess whether a lightweight model's answer is reliable (>0.96 threshold) before triggering more expensive models in the cascade. * Complexity Classification: **?**] employs a T5-Large classifier to detect query complexity across three tiers (No Retrieval, Single-step, and Multi-step), allowing the system to "smoothly oscillate" between strategy depths. * Preference-Learned Routing: **?**] analyzes human preferences via the Chatbot Arena dataset to train Matrix Factorization and SW Ranking routers that predict the probability of a strong model outperforming a weak model. * Contrastive RAG-Aware Routing: Zhang et al. [33] introduces RAGRouter, which uses contrastive learning and RAG capability embeddings to model how retrieved documents dynamically reshape an LLM's ability to answer, rather than relying on static parametric profiles. * Source-Specific Routing: Guo et al. [31] implements an LLM-as-a-knowledge-router that utilizes source-visible profile metadata (e.g., SQL, RAG corpus, API) to dispatch sub-queries to the most appropriate heterogeneous data silo.

* Identified Performance-Quality Trade-offs: * **?**] identifies a near-Full Accuracy equilibrium, where routers recover 95performance gap between weak and strong models while maintaining the aforementioned cost savings. * Xu et al. [30] trades off the local compute cost of a 3B SLM decomposer for a 14.2gain on complex multi-hop QA tasks compared to standard RAG baselines. * **?**] shows that while the multi-step approach is more accurate, it is significantly more expensive; their adaptive framework achieves the "best of both worlds" by matching multi-step accuracy on complex queries while retaining the efficiency of single-step models on trivial ones.

* Addressing Enterprise Constraints: * Heterogeneous Silos: Guo et al. [31] addresses the constraint of unmergeable knowledge sources (SQL vs. Unstructured RAG) by using a routing layer to prevent "context pollution" and manage silos without requiring schema unification. * Fixed API Budgets: Chen et al. [2] and **?**] explicitly model budget-aware optimization, where the routing threshold α can be adjusted to enforce strict cost constraints or bias towards high-quality outputs based on remaining token credits.

Section 3: Comparative Analysis & Synthesis Opportunities

* Contrast in Routing Foundations: While **?**] relies on human preference data and battle labels from Chatbot Arena to determine routing, **?**] utilizes automated silver labels derived from model outcomes and intrinsic dataset biases (single-hop vs. multi-hop) to train their complexity classifier. * Contrast in Knowledge Modeling: Zhang et al. [33] argues against the "Static Knowledge Assumption" found in earlier routers (like those in FrugalGPT), proving that routing decisions must explicitly account for the interaction between retrieved documents and the model via a cross-encoder rather than just query-model compatibility [33, 2]. * Differences in Task Specialization: Guo et al. [31] and Zhao et al. [5] focus on the heterogeneity of the knowledge base (routing to SQL, JSON, or APIs), whereas Xu et al. [30] and **?**] focus on the complexity of the reasoning chain (routing between decomposition, iterative loops, and single-shot responses). * Synthesis of Quality Improvements: Xu et al. [30] reports

that a fine-tuned 3B SLM decomposer can surpass a frozen 32B model in planning quality, which aligns with **?**]'s finding that small, specialized classifiers (T5-Large) are sufficient for routing decisions, suggesting that enterprise RAG optimization does not require large-scale models for the "control" layer. * Evaluation Dataset Variation: Contrast Chen et al. [2]'s use of domain-specific enterprise datasets (HEADLINES for finance, OVERRULING for legal) with the reliance of **?**], Zhang et al. [33], and Xu et al. [30] on standard academic benchmarks (PopQA, NQ, HotpotQA) to validate their routing frameworks.

3.2 Small-to-Big retrieval

3.3 Semantic Re-ranking

3.4 Prompt Caching

4 Methods

4.1 Architecture of research program

4.2 Research methodology

Oisko tästä datasetiksi https://arxiv.org/pdf/2403.12031?

5 Results

TODO

6 Discussion

The idea of discussion is to bring up points that you noticed during research. As a chapter, this is the most freeform one. A good way to discuss the points is first to write up subtitles with clear statements and then expand on those. Discussion points can occasionally be also in a question format or refer to a larger context beyond the thesis to bring in ideas from other papers to consider. In case your work has limitations that you can observe, it can be a good idea to write a separate section considering those.

7 Conclusion

The idea of the conclusion is the opposite of the introduction. Here we literally bring the reader out of the topic, not in. A good way to write conclusion is to start by reminding the reader of your research question(s) and then state how your work addressed them. To finish your conclusion, consider open research questions (this can be a list for example).

References

[1] Stanford University Human-Centered Artificial Intelligence Institute. The Artificial Intelligence Index Report 2026. Technical report, Stanford University, April 2026. URL https:///hai.stanford.edu/ai-index/2026-ai-index-report.

[2] Lingjiao Chen, Matei Zaharia, and James Zou. Frugalgpt: How to use large language models while reducing cost and improving performance. *arXiv preprint arXiv:2305.05176*, 2023.

[3] Google Cloud. *Vertex AI Pricing: Gemini Models*, 2026. URL https://cloud.google.com/vertex-ai/generative-ai/pricing. Accessed June 2026.

[4] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in Neural Information Processing Systems*, 33:9459–9474, 2020.

[5] Siyun Zhao, Yuqing Yang, Zilong Wang, Zhiyuan He, Luna K. Qiu, and Lili Qiu. Retrieval augmented generation (rag) and beyond: A comprehensive survey on how to make your llms use external data more wisely. *arXiv preprint arXiv:2409.14924*, 2024.

[6] Shangyu Wu, Ying Xiong, Yufei Cui, Haolun Wu, Can Chen, Ye Yuan, Lianming Huang, Xue Liu, Tei-Wei Kuo, Nan Guan, and Chun Jason Xue. Retrieval-augmented generation for natural language processing: A survey. *arXiv preprint arXiv:2407.13193*, 2024.

[7] Broadcom Inc. Spring ai. https://spring.io/projects/spring-ai, 2026. Accessed: 2026-06-24.

[8] Elias Lumer, Matt Melich, Olivia Zino, Elena Kim, Sara Dieter, Pradeep Honaganahalli Basavaraju, Vamse Kumar Subbiah, James A. Burke, and Roberto Hernandez. Rethinking retrieval: From traditional retrieval augmented generation to agentic and non-vector reasoning systems in the financial domain for large language models. *arXiv preprint arXiv:2511.18177*, 2025.

[9] Yue Yu, Wei Ping, Zihan Liu, Boxin Wang, Jiaxuan You, Chao Zhang, Mohammad Shoeybi, and Bryan Catanzaro. Rankrag: Unifying context ranking with retrieval-augmented generation in llms. In *38th Conference on Neural Information Processing Systems (NeurIPS 2024)*, 2024.

[10] In Gim, Guojun Chen, Seung-seob Lee, Nikhil Sarda, Anurag Khandelwal, and Lin Zhong. Prompt cache: Modular attention reuse for low-latency inference. In *Proceedings of the 5th MLSys Conference*, 2024.

[11] American Bar Association. Bc tribunal confirms companies remain liable for information provided by ai chatbot. https://www.americanbar.org/groups/business_law/resources/business-law-today/2024-february/bc-tribunal-confirms-companies-remain-liable-information-provided-ai-chatbot/, February 2024. Accessed: 2026-06-24.

[12] Stuart J. Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach*. Pearson Education, Inc., Upper Saddle River, New Jersey, third edition, 2010. ISBN 978-0-13-604259-4.

[13] Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Zican Dong, Yupeng Hou, Beichen Zhang, Yingqian Min, Junjie Zhang, Peiyu Liu, et al. A survey of large language models. *Frontiers of Computer Science*, 20(12):2012627, 2026.

[14] Alec Radford, Jeff Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language Models are Unsupervised Multitask Learners. 2019.

[15] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017*, pages 5998–6008, Long Beach, CA, USA, December 4-9 2017. Neural Information Processing Systems (NIPS).

[16] Rishi Bommasani, Drew A Hudson, Ehsan Adeli, Russ Altman, Simran Arora, Sydney von Arx, Michael S Bernstein, Jeannette Bohg, Antoine Bosselut, Emma Brunskill, et al. On the opportunities and risks of foundation models. *arXiv preprint arXiv:2108.07258*, 2021.

[17] Tristan Coignion, Clément Quinton, and Romain Rouvoy. A Performance Study of LLM-Generated Code on Leetcode. In *Proceedings of the 28th International Conference on Evaluation and Assessment in Software Engineering*, EASE 2024, page 79–89. ACM, June 2024. doi: 10.1145/3661167.3661221.

[18] Numa. Dhamani. *Introduction to Generative AI*. Manning Publications Co. LLC, New York, 1st ed. edition, 2024. ISBN 9781638354345.

[19] Nelson F Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the association for computational linguistics*, 12: 157–173, 2024.

[20] Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Ye Jin Bang, Andrea Madotto, and Pascale Fung. Survey of hallucination in natural language generation. *ACM computing surveys*, 55(12):1–38, 2023.

[21] Gemini Team, Google. Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context. Technical report, Google DeepMind, 2024. URL https://arxiv.org/abs/2403.05530.

[22] Google DeepMind. Gemini 3 Flash Model Card. Model card, Google, December 2025. URL https://storage.googleapis.com/deepmind-media/Model-Cards/Gemini-3-Flash-Model-Card.pdf.

[23] Bhuwan Dhingra, Jeremy R Cole, Julian Martin Eisenschlos, Daniel Gillick, Jacob Eisenstein, and William W Cohen. Time-aware language models as temporal knowledge bases. *Transactions of the Association for Computational Linguistics*, 10:257–273, 2022.

[24] Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yixin Dai, Jiawei Sun, Haofen Wang, Haofen Wang, et al. Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997*, 2(1):32, 2023.

[25] Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with gpus. *IEEE transactions on big data*, 7(3):535–547, 2019.

[26] Kurt Shuster, Spencer Poff, Moya Chen, Douwe Kiela, and Jason Weston. Retrieval augmentation reduces hallucination in conversation. In *Findings of the Association for Computational Linguistics: EMNLP 2021*, pages 3784–3803, 2021.

[27] Bo Ni, Zheyuan Liu, Leyao Wang, Yongjia Lei, Yuying Zhao, Xueqi Cheng, Qingkai Zeng, Luna Dong, Yinglong Xia, Krishnaram Kenthapadi, et al. Towards trustworthy retrieval augmented generation for large language models: A survey. *arXiv preprint arXiv:2502.06872*, 2025.

[28] Soyeong Jeong, Jinheon Baek, Sukmin Cho, Sung Ju Hwang, and Jong C Park. Adaptive-rag: Learning to adapt retrieval-augmented large language models through question complexity. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 7036–7050, 2024.

[29] Isaac Ong, Amjad Almahairi, Vincent Wu, Wei-Lin Chiang, Tianhao Wu, Joseph E Gonzalez, M Waleed Kadous, and Ion Stoica. Routellm: Learning to route llms with preference data. *arXiv preprint arXiv:2406.18665*, 2024.

[30] Ran Xu, Wenqi Shi, Yuchen Zhuang, Yue Yu, Joyce C. Ho, Haoyu Wang, and Carl Yang. Collab-RAG: Boosting retrieval-augmented generation for complex question answering via white-box and black-box LLM collaboration. *arXiv preprint arXiv:2504.04915*, 2025.

[31] Minghao Guo, Qingcheng Zeng, Xujiang Zhao, Yanchi Liu, Wenchao Yu, Mengnan Du, Haifeng Chen, and Wei Cheng. Deepsieve: Information sieving via LLM-as-a-knowledge-router. In *Findings of the Association for Computational Linguistics: EACL 2026*, pages 3054–3077, 2026.

[32] Jiajie Jin, Yutao Zhu, Zhicheng Dou, Guanting Dong, Xinyu Yang, Chenghao Zhang, Tong Zhao, Zhao Yang, and Ji-Rong Wen. Flashrag: A modular toolkit for efficient retrieval-augmented generation research. In *Companion of the 16th ACM/SPEC International Conference on Performance Engineering (WWW Companion '25)*. ACM, 2025.

[33] Jiarui Zhang, Xiangyu Liu, Yong Hu, Chaoyue Niu, Fan Wu, and Guihai Chen. Ragrouter: Learning to route queries to multiple retrieval-augmented language models. In *39th Conference on Neural Information Processing Systems (NeurIPS 2025)*, 2025.